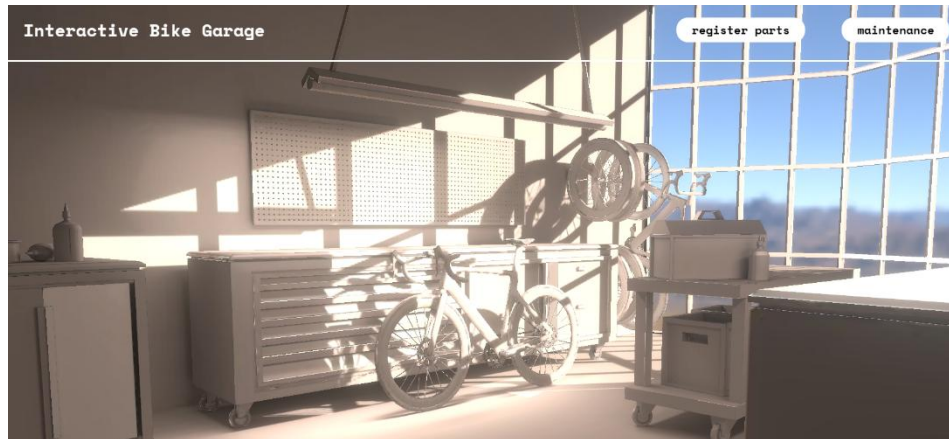


Artist Statement, Interactive Bike Garage II
By Émile Bédard 40343664

Work presented to Sabine Rosenberg
CART 263, Creative Computation II, Section A
Computation Arts, Concordia University — April 12th, 2026

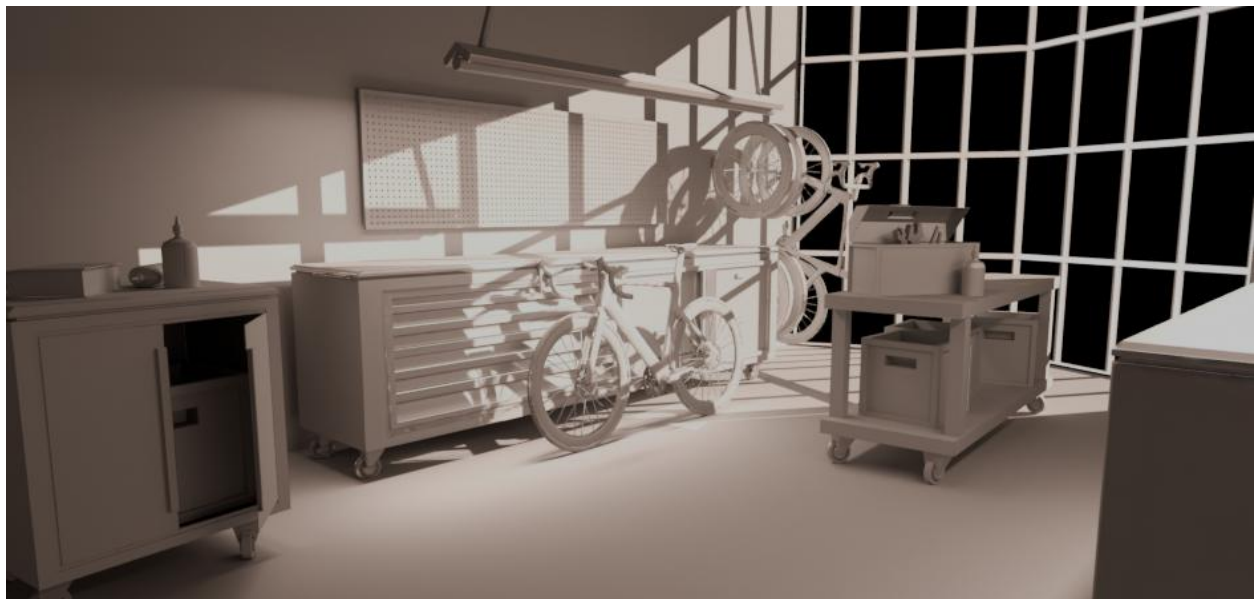
URL of the project (desktop and mobile):

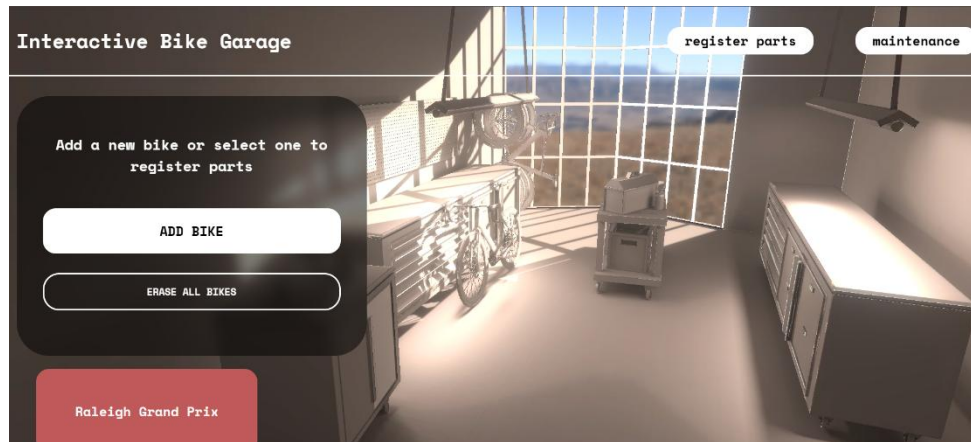
https://emilebedard.github.io/cart263/Projects/project2/interactive_bike_garage_ii/



Main landing page

The main atmosphere of the web experience was the whole idea of the second project: offer an immersive 3D modern scene of a bicycle workshop. I modeled everything except for the bike and decided on the layout and composition of the scene based on my own preferences. I would like to imagine this space as my own future workshop.





Layout and HTML hierarchy

My next challenge was to keep as much as possible from my previous hmtl/css layout and maximize my time and energy on the 3D with Three.js. I still feel not completely sure about the styling but integrating the 3D canvas seamlessly was not easy. For the next version, I will start from scratch and build on top of the canvas to really make a new styling and UI interface that feels much more integrated and cohesive.

```
function addMeshes(loaderObjArray) {
  console.log(loaderObjArray);
  //garage
  const garageScene = loaderObjArray[0].scene; //load blender data and place in the scene
  scene.add(garageScene);

  //cabinet
  const cabinetObj = loaderObjArray[1].scene; //load blender data and place in the scene
  scene.add(cabinetObj);

  //workbench
  const workbenchObj = loaderObjArray[2].scene; //load blender data and place in the scene
  scene.add(workbenchObj);

  ///camera from blender---
  let blenderCamera = garageScene.children.find(child => child.isCamera);

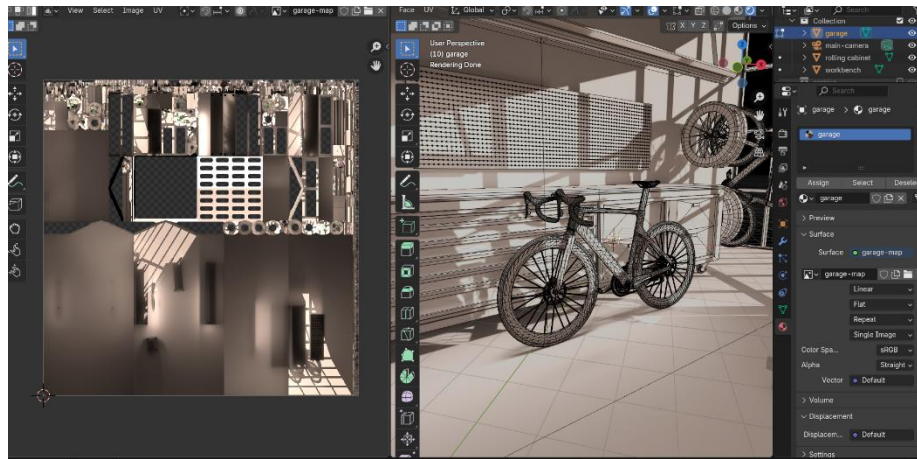
  //change between initial and blender cam here
  let usedCam = blenderCamera;
  if (usedCam === blenderCamera) {
    blenderCamera.name = "blenderCamera";
    camera = blenderCamera;
  }

  const isRegistryPage = window.location.pathname.includes('parts_registry.html');
  if (isRegistryPage) {
    const registryPos = new THREE.Vector3(-17, 15, 40);
    camera.position.copy(registryPos);
    originalBlenderPos.copy(registryPos);
  } else {
    originalBlenderPos.copy(blenderCamera.position);
  }

  // Update the aspect ratio to match the current html container dimensions stated in the setup container function
  camera.aspect = sizes.width / sizes.height;
  camera.updateProjectionMatrix();
  renderPass.camera = camera;
}
```

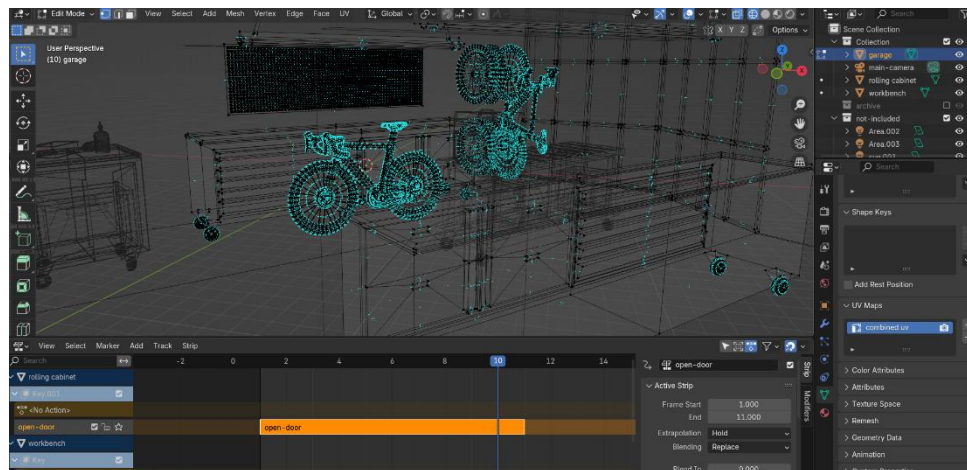
Importing and integrating GLTFs in THREE.JS

Importing the GLTF format and retrieving data, animation, scenes, lights and cameras was very tricky. It was imperative to keep a good file system and to really organize my objects name to be able to point them out easily afterwards. The camera logic was especially delicate because I needed to redirect it from the Three.Js interface to store it in a variable, rename it and then attach to my renderer not only once it was loaded from my try/catch asynchronous function.



Lightmap baking and UV unpacking

Lightmap baking and UV unpacking were familiar concept for me with the previous experience that I had with Blender, but I never fully implemented them in a project before this one. It was tricky to figure out the technical manipulations and procedures but once these constraints were out the way, I really had a lot of fun to play with the light and achieve a much more realistic look for my scene.



Normals and face orientation

Finally, normals and 3D face orientations were also a significant speed bump in my Blender 3D Pipeline. As I explain in the video, importing external 3D models and assets can always be tricky and this bicycle 3D model was no exception. I had to rework all the faces orientation manually to achieve realistic light reflections with the rendering and global illumination.